

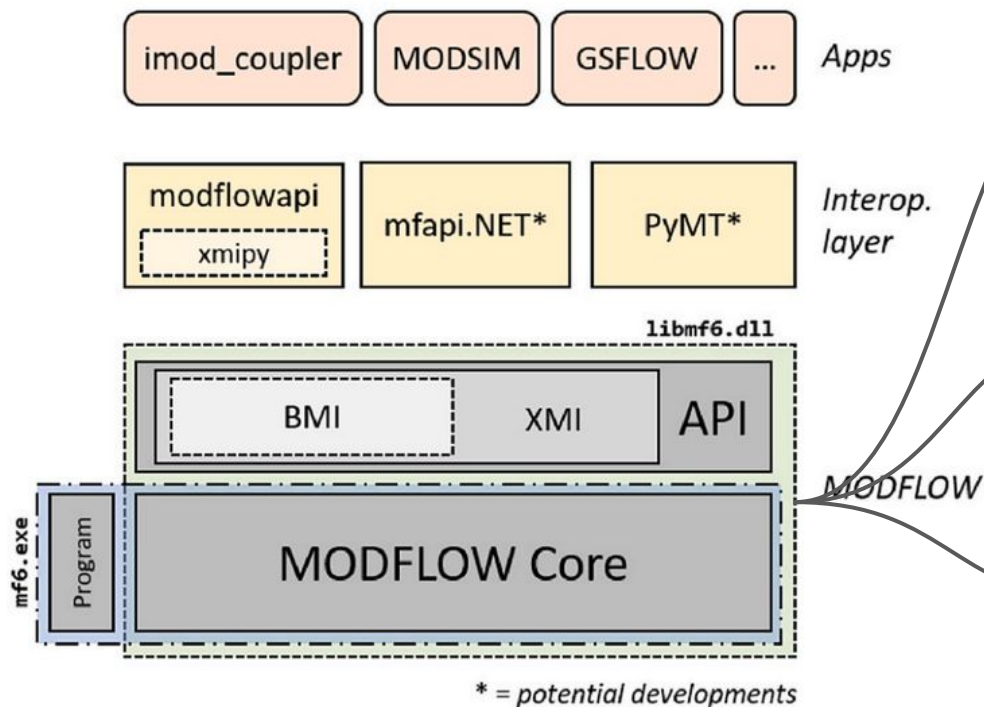
MODFLOW 6 API

Luis M. de la Cruz Salas
Depto. de Recursos Naturales
28 de julio de 2025

Introducción

- La API de MODFLOW permite que otros programas controlen este software en tiempo de ejecución y modifiquen interactivamente variables sin tener que acceder al código fuente.
- Está basado en *Basic Model Interface* ([BMI](#)), que permite el acoplamiento con otros modelos (por ejemplo [GSFLOW](#), [MODSIM](#)).
- La capacidad de modificar variables en cada paso de tiempo requirió de un *eXtended Model Interface* ([XMI](#)).
- El API es muy general de tal manera que en principio permite acoplar MODFLOW con cualquier otro programa.

MODFLOW API



El API (**XMI**, *eXtended Model Interface*) se compone de todas las funcionalidades de **BMI** (*Basic Model Interface*) más algunas extensiones necesarias para controlar la simulación a un detalle más fino.

Basic Model Interface (BMI): conjunto de funcionalidades que definen cómo inicializar (*initialize*) una simulación, actualizar el estado del modelo y avanzar un paso de tiempo (*update*) y finalizar la ejecución (*finalize*).

MODFLOW core:

- `mf6.exe` o
- `libmf6.dll` (.so, .dylib)

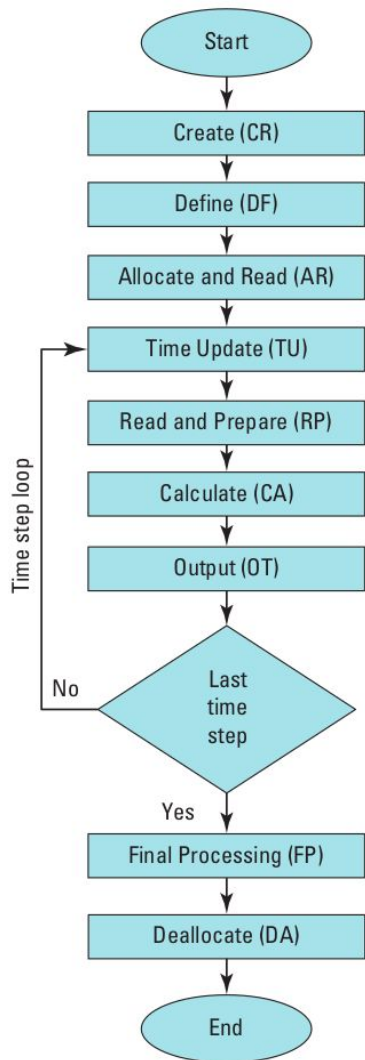
Ambas construcciones son equivalentes y proporcionan los mismos resultados.

BMI en MODFLOW

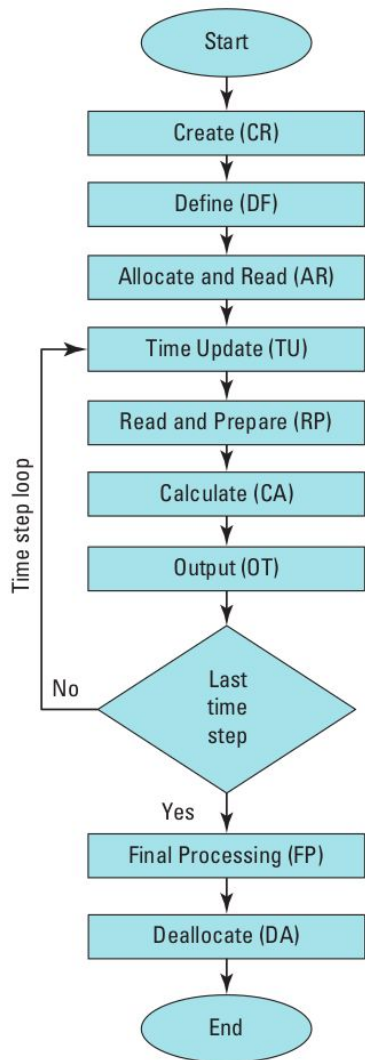
Simulación (inicialización)

The main program: Ejecuta varios *primary procedures*¹ en la secuencia apropiada para realizar y controlar la simulación.

- CR: Crea objetos de model, package, exchange y solution.
- DF: Define algunos atributos de los objetos, por ejemplo el tamaño de algunos arreglos.
- AR: Genera los arreglos. También lee información que será constante durante toda la simulación.



¹ *Primary procedure*: compuesto de varias *primary routines*, las cuales son el código que implementa el procedimiento para una componente (métodos de la clase).

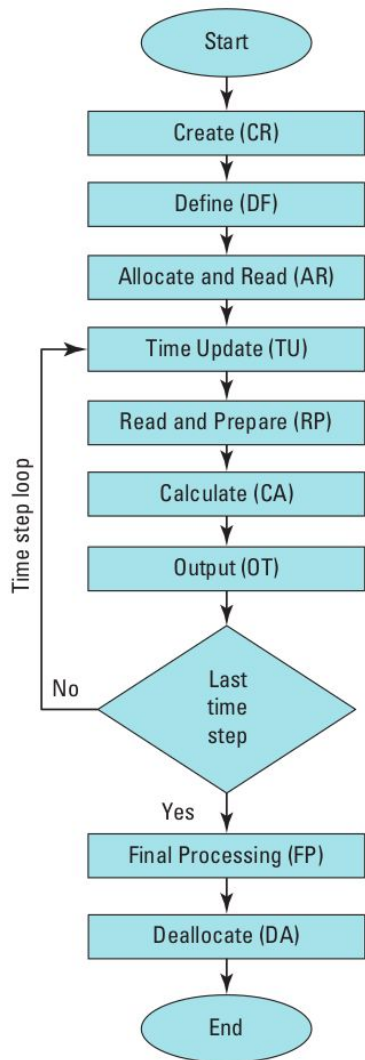


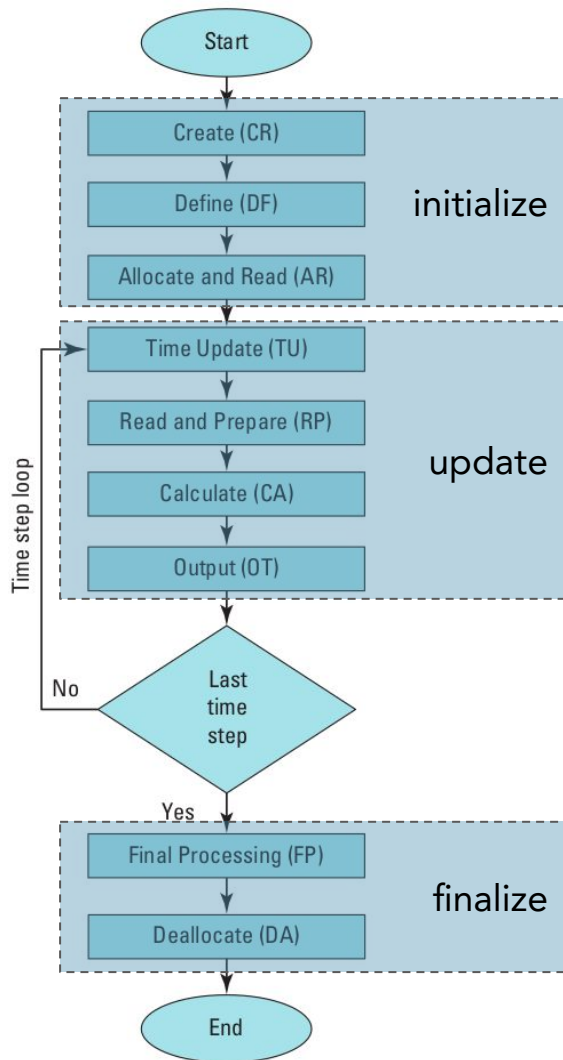
Simulación (actualización)

- TU: Incrementa las variables del tiempo y calcula la longitudes del paso de tiempo.
- RP: Lee la información de los archivos de entrada, conforme se necesite, para actualizar los estreses hidrológicos u otras entradas que varían con el tiempo.
- CA: Actualiza las variables dependientes. Para soluciones numéricas, utiliza métodos iterativos para resolver sistemas de ecuaciones no lineales.
- OT: Escribe los resultados de la simulación en archivos para cada paso de tiempo o como sea requerido.

Simulación (finalización)

- FP: Escribe mensajes de terminación y cierra todos los archivos.
- DA: Desaloja toda la memoria.





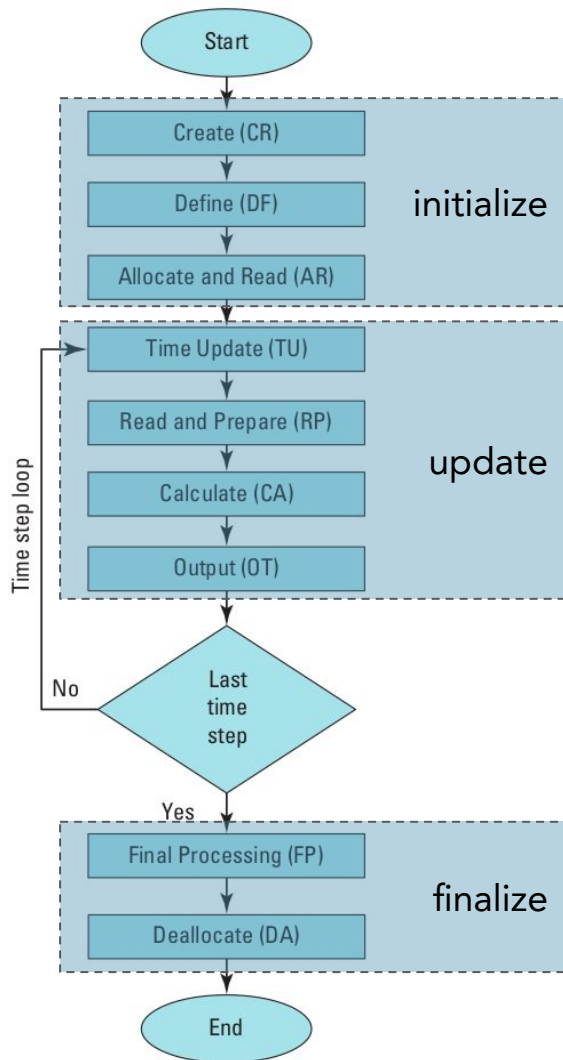
El API agrega las subrutinas de MODFLOW en las unidades funcionales BMI (*initialize*, *update*, *finalize*) que son las que controlan la simulación.

MODFLOW es el responsable de implementar el ciclo de tiempo.

Recordemos que la simulación en MODFLOW se divide en:

- periodos de estrés: intervalos durante los cuales se redefinen las condiciones de estrés, y
- pasos de tiempo de cada periodo de estrés: constituyen los pasos de tiempo, que pueden ser no equidistantes, para resolver los modelos numéricos de la simulación.

Obsérvese que no hay un ciclo explícito sobre los periodos de estrés; de esta manera se cumple con la interpretación del ciclo de tiempo de BMI.



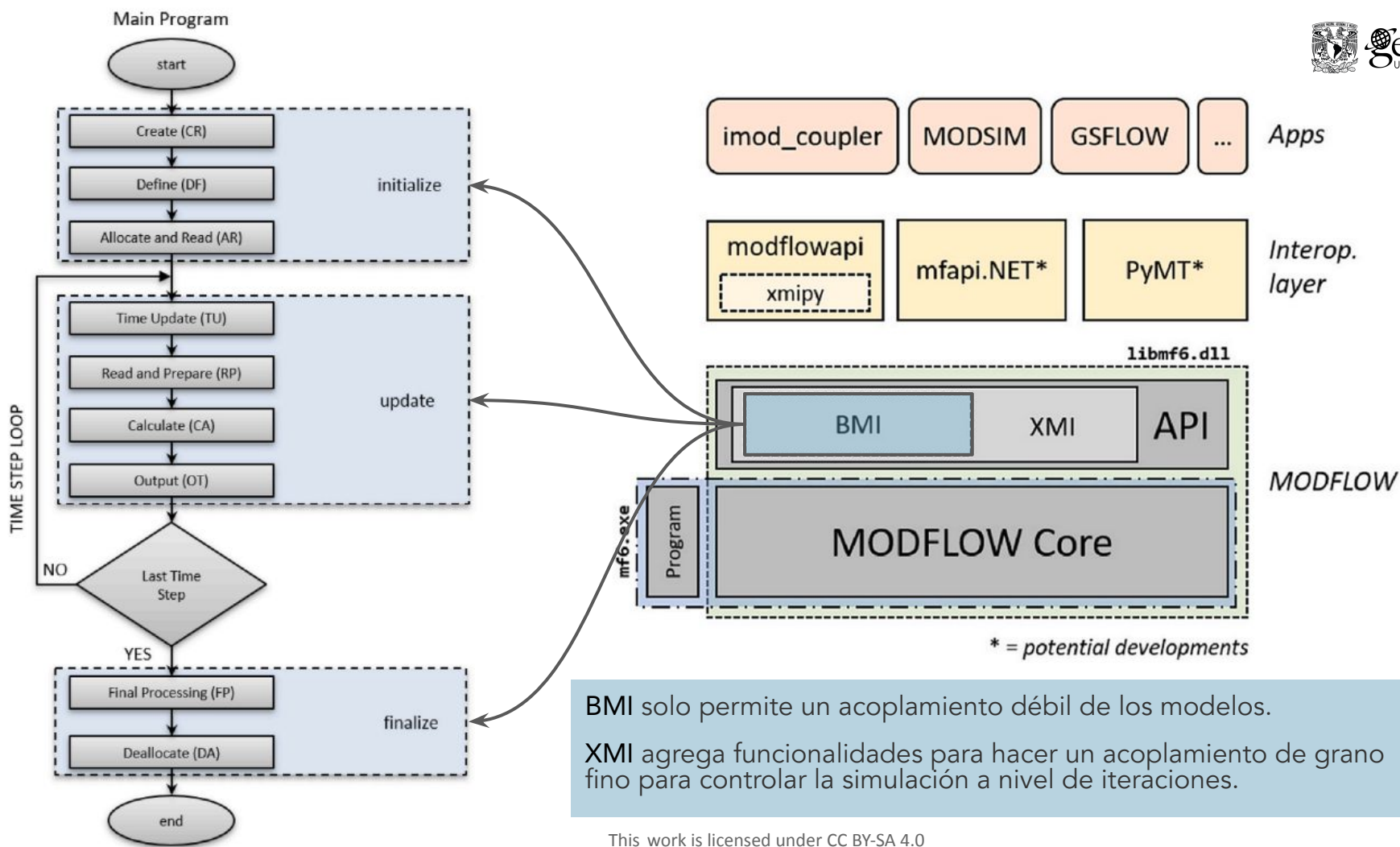
update avanza la simulación en un paso de tiempo.

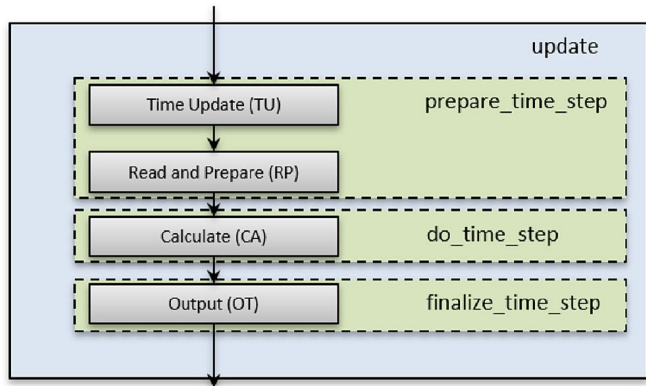
- El tiempo inicial se fija en 0.0 para todas las simulaciones, se obtiene con **get_start_time()**.
- El tiempo final es la suma de todos los pasos de tiempo, se obtiene con **get_end_time()**.
- El paso de tiempo actual se obtiene con **get_current_time()**, este valor se actualiza hasta que se ejecuta TÚ.
- Longitud del paso de tiempo actual: **get_time_step()**.

Acceso a los datos de los modelos.

- La mayoría de las variables (escalares/arreglos) en MODFLOW son punteros y tienen una etiqueta única: nombre de la variable, nombre de la componente y paquete de MODFLOW al que pertenece.
- La función **get_var_address()** construye esta etiqueta. Requiere: (**var_name: str, component_name: str, subcomponent_name: str = ""**).
 - Por ejemplo: **get_var_address("XC", "FLOW", "DIS")** regresa 'FLOW/DIS/XC'.
- Una lista de todas las variables se puede obtener con la función **get_input_var_names()** ◦ **get_output_var_names()**.

eXtended Model Interface (XMI)



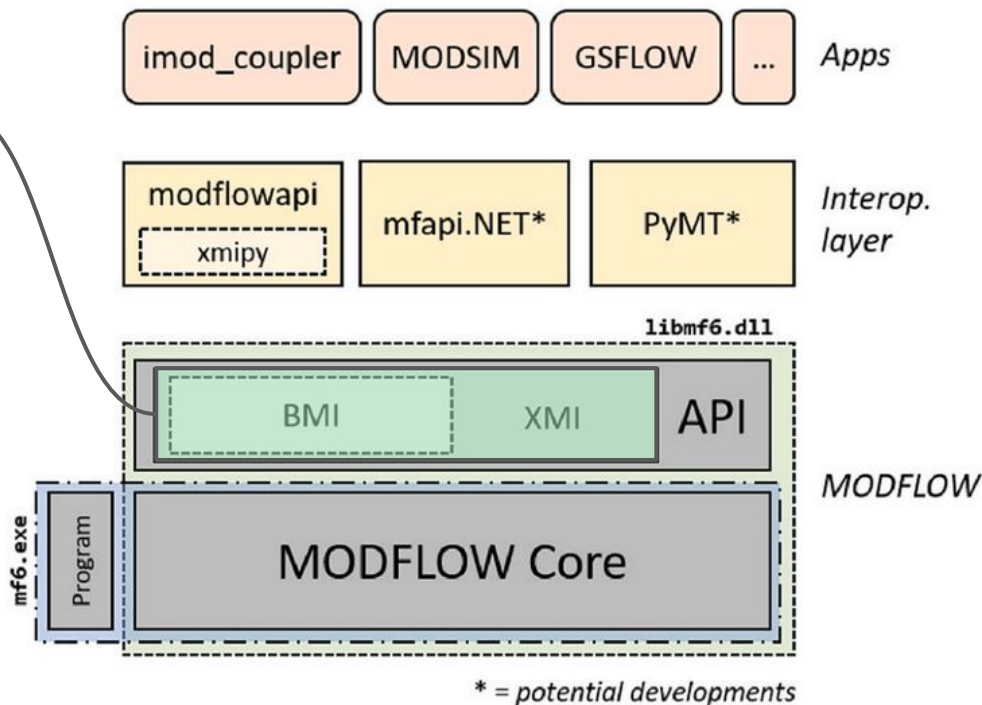


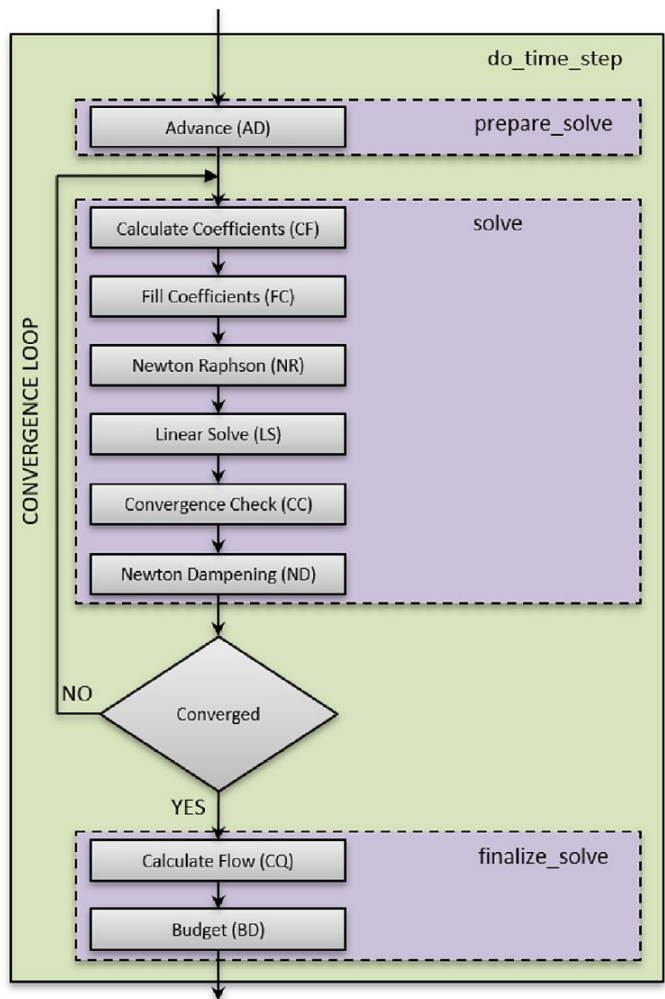
update se subdivide en unidades más pequeñas para permitir la modificación de datos que varían con el tiempo.

Los datos se leen de archivos en cada paso de tiempo (RP).

En prepare_time_step se leen los datos antes de ejecutar do_time_step.

Estos datos pueden modificarse usando las funciones de **BMI**.





XMI también extiende `do_time_step`, lo cual permite obtener/modificar variables durante el ciclo de solución numérica.

XMI está diseñado como una biblioteca estándar de lenguaje C.

Esto permite integrar el API desde programas en lenguajes como C++, Python, Java, C#.NET, e incluso ejecutar sus funcionalidades desde Jupyter notebooks.

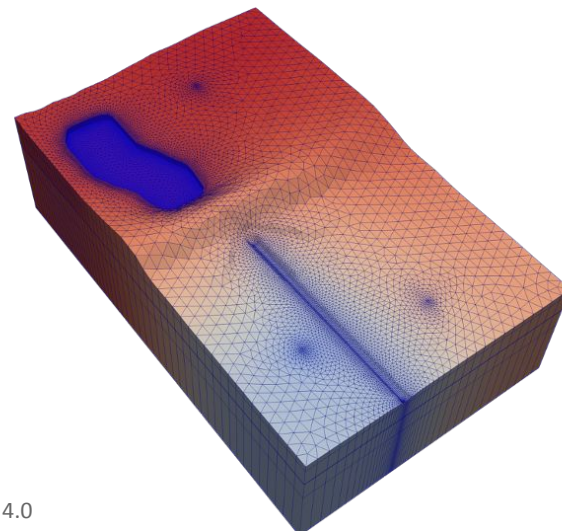
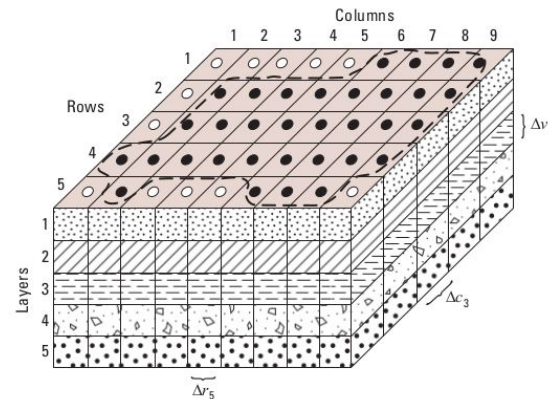
La biblioteca `xmipy` gestiona la interoperabilidad entre BMI y XMI, y encapsula la complejidad de los tipos de datos entre Python y Fortran.

La biblioteca `modflowapi` extiende `xmipy` para crear una interface amigable al usuario.

MODFLOW 6

MODFLOW 6: un vistazo

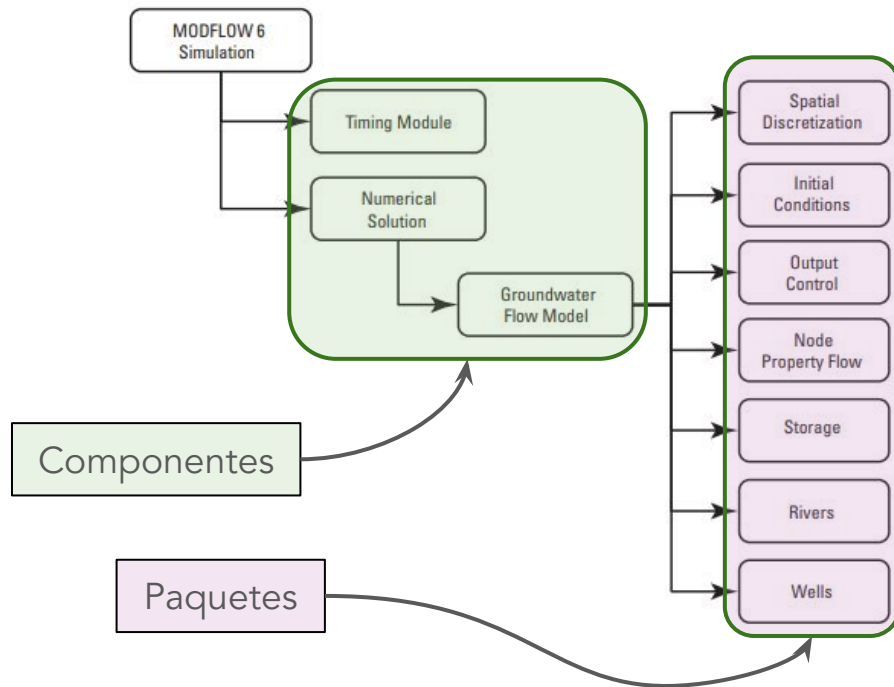
- Los modelos numéricos se basan en el método CVFD (*Control-volume Finite-Difference*) en donde cada celda puede estar hidráulicamente conectada con cualquier número de celdas vecinas.
 - Malla regular: capas, renglones y columnas.
 - Malla en capas definida por conjuntos de vértices (x, y).
 - Mallas no estructuradas generales.



Componentes.

- *Simulation, Timing, Solutions, Models, Exchange, Utilities.*
 - El *Timing Module* calcula el paso de tiempo y se aplica para todos los modelos de la simulación.
 - *Numerical Solution* se encarga de configurar el solucionador para resolver las ecuaciones de uno o más modelos y de los intercambios.
 - Un modelo es una componente principal de la simulación y representa un proceso hidrológico

Jerarquía de la simulación



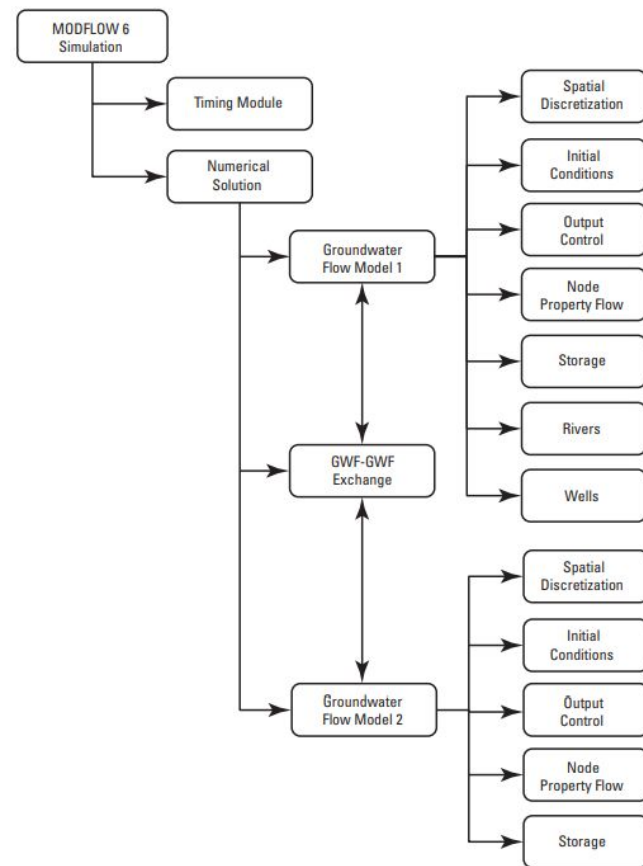
Paquetes.

- Está dividido en paquetes; cada paquete se encarga de un aspecto de la simulación.
 - Cálculo interno del flujo de agua subterránea.
 - Esfuerzos.
 - Esfuerzos avanzados.
 - Salidas del modelo.

Package Name	Abbreviation	Package Category
Discretization	DIS, DISV, or DISU	Hydrologic/Internal
Initial Conditions	IC	Hydrologic/Internal
Node Property Flow	NPF	Hydrologic/Internal
Horizontal Flow Barrier	HFB	Hydrologic/Internal
Ghost Node Correction	GNC	Hydrologic/Internal
Storage	STO	Hydrologic/Internal
Specified Head	CHD	Hydrologic/Stress
Well	WEL	Hydrologic/Stress
Recharge	RCH	Hydrologic/Stress
River	RIV	Hydrologic/Stress
General-Head Boundary	GHB	Hydrologic/Stress
Drain	DRN	Hydrologic/Stress
Evapotranspiration	EVT	Hydrologic/Stress
Stream-Flow Routing	SFR	Hydrologic/Advanced Stress
Lake	LAK	Hydrologic/Advanced Stress
Multi-Aquifer Well	MAW	Hydrologic/Advanced Stress
Unsaturated Zone Flow	UZF	Hydrologic/Advanced Stress
Water Mover	MVR	Hydrologic/Advanced Stress
Model Observations	OBS	Output
Output Control	OC	Output

Varios modelos

- Cuando hay más de un modelo, se puede usar *Exchange* para conectarlos; se usa la etiqueta GWF-GWF o GWF-GWT.
- Se pueden definir tantos intercambios como sean necesarios para conectar varios modelos.
- El acoplamiento de modelos se hace a nivel de matriz, lo cual permite resolverlos de manera eficiente, como si solo fuera un modelo en una malla no estructurada.



Definición de parámetros de la simulación

```
init = {  
    'sim_name' : "flow",  
    'exe_name' : "C:\\ ... \\mf6",  
    'sim_ws' : "test1"  
}
```

`o_sim = flopy.mf6.MFSimulation( )`

```
tdis= {  
    'units': "DAYS",  
    'nper' : 1,  
    'perioddata': [(1.0, 2, 1.0)]  
}
```

`o_tdis = flopy.mf6.ModflowTdis( )`

```
ims = {}
```

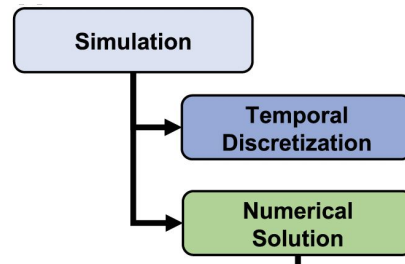
`o_ims = flopy.mf6.ModflowIms( )`

Flopy + xmf6

Inicialización de la simulación

```
o_sim = xmf6.common.init_sim(init = init, tdis = tdis,  
                               ims = ims, silent = False)
```

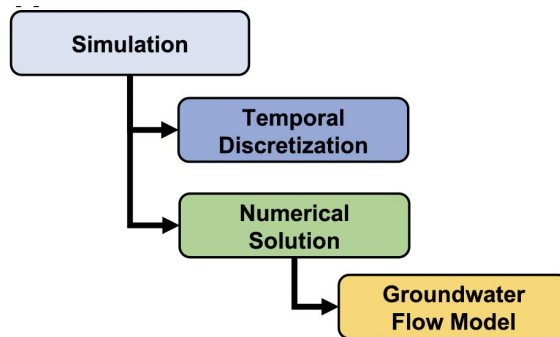
- Inicializa las componentes de la simulación.
 - `o_sim`
 - `o_tdis`
 - `o_ims`
- Crea el directorio de trabajo (working space).



Definición de los parámetros del modelo

```
gwf = {
    'modelname': init["sim_name"],
    'model_nam_file': f"{init["sim_name"]}.nam",
    'save_flows': True
}
```

} `o_gwf = flopy.mf6.ModflowGwf()`



Definición de parámetros de los paquetes

```
dis = {  
    'length_units': "meters",  
    'nlay': 1,  
    'nrow': 1,  
    'ncol': 5,  
    ...  
}  
  
ic = {'strt': 1.0 }  
  
chd = {  
    'stress_period_data': chd_data,  
}  
  
...
```

} `o_dis = flopy.mf6.Modflowdis( )`

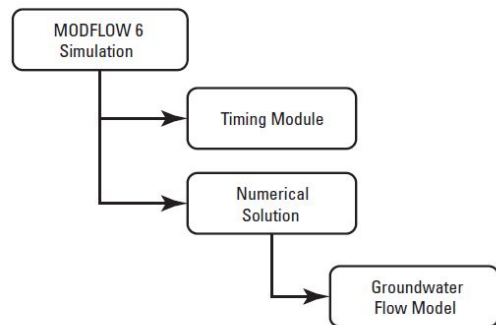
} `o_ic = flopy.mf6.Modflowic( )`

} `o_chd = flopy.mf6.Modflowchd( )`

Paquetes

```
o_gwf, packages = xmf6.gwf.set_packages(o_sim, silent = True,  
    gwf = gwf, dis = dis, ic = ic, chd = chd, npf = npf, oc = oc)
```

- Agrega los paquetes al modelo.



Escritura de archivos de entrada

```
o_sim.write_simulation(silent = True)
```

- Genera todos los datos de entrada para la simulación:
 - mfsim.nam
 - flow.nam
 - flow.tdis
 - flow.ims

Paquetes

- flow.dis
- flow.chd
- flow.ic
- flow.npf
- flow.oc

Ejemplos en: [mf6 tutorial](#)

Usando la API

API

Para realizar y controlar una simulación de MODFLOW con la API se requiere crear un objeto de la clase **ModflowApi**:

```
from modflowapi import ModflowApi  
mf6 = ModflowApi("libmf6.dll", working_directory = ...)
```

Este objeto tendrá acceso a todos los métodos de la API.

Inicialización de la API

mf6.initialize(mf6_config_file)

- El modelo base de MODFLOW se inicializa a partir del archivo **mfsim.nam**. Esta función genera los siguientes archivos:
 - flow.bud (vacíos)
 - flow.dis.grb
 - flow.hds (vacíos)
 - flow.lst
 - mfsim.lst
 - mfsim.stdout
- Con esta función se inicializa la simulación, se leen todos los datos estáticos del modelo y a partir de ahora se tiene acceso a algunas variables de la simulación.

Ejemplos

Software requerido

- Instalar todo el ambiente de Python con las bibliotecas comunes (numpy, matplotlib, pandas, jupyter lab, etc).
- Obtener MODFLOW 6
 - [MODFLOW 6 Repo](#)
 - [MODFLOW-nightly-build](#)
- Instalar [flopy](#):
pip install flopy
- Instalar [modflowapi](#):
pip install modflowapi
- Instalar la biblioteca [xmf6](#):
pip install git+https://github.com/luiggix/xmf6
- Clonar el repositorio [mf6 tutorial](#) de ejemplos de esta presentación ejecutar el siguiente comando:
git clone https://github.com/luiggix/mf6_tutorial.git

Ejemplos del artículo (Hughes, 2021)

- Enlace a los datos [emsdataset](#) (emsdatasets.zip)
- Desempacar el archivo para obtener lo siguiente:

emsdataset/	ModflowPRMS/
bin/ ←	ModflowMsw/
common/	Modsim/
CustomPackage/	OptimizeGroundwater/

Colocar aquí la biblioteca compartida: libmf6.dll

NOTA: en algunos ejemplos se usa: `np.float` que ya está descontinuado de numpy, cambiar por `np.float64`.

Referencias

- Hughes, J. D., Russcher, M. J., Langevin, C. D., Morway, E. D., & McDonald, R. R. (2021). The MODFLOW Application Programming Interface for simulation control and software interoperability. *Environmental Modelling & Software*, 148, 105257. <https://doi.org/10.1016/j.envsoft.2021.105257>.
- Langevin, C.D., Hughes, J.D., Banta, E.R., Niswonger, R.G., Panday, Sorab, and Provost, A.M., 2017, Documentation for the MODFLOW 6 Groundwater Flow Model: U.S. Geological Survey Techniques and Methods, book 6, chap. A55, 197 p., <https://doi.org/10.3133/tm6A55> .
- Hughes, J.D., Langevin, C.D., and Banta, E.R., 2017, Documentation for the MODFLOW 6 framework: U.S. Geological Survey Techniques and Methods, book 6, chap. A57, 40 p., <https://doi.org/10.3133/tm6A57> .